

SYNDICATE BY MAXIMOR • TRACK 2 – AUTONOMOUS OFFICE OF THE CFO

TieOut

The close agent that refuses to post anything it cannot prove.

Track Autonomous Office of the CFO

Team [Team name – 2 builders]

Built with Agent Orchestrator

EVERY PAYMENT CYCLE, EVERY COMPANY

One deposit hides hundreds of transactions

When a payment processor pays out, it sends one net number — never the gross sales figure. Someone has to break it back apart and post every piece to the right account before the books can close.

56%

STILL RECONCILE IN SPREADSHEETS

94%

OF THOSE MISS DEADLINES

PAYOUT • DODO PAYMENTS • SETTLED

\$9,412.33

Revenue — 58%

Refunds — 14%

Processing fees — 10%

Tax remitted — 9%

Disputes — 9%

THIS IS NOT A BANK-RECONCILIATION PROBLEM

Four things spreadsheets can't hold

STRUCTURAL

Many-to-one by design

One payout bundles hundreds of charges, fees, and refunds. VLOOKUP-style matching breaks down well before real transaction volume.

TIME-DISTRIBUTED

Disputes are 3+ events

A chargeback is an initial debit, a possible reversal weeks later, and a non-refundable fee either way — not one line item.

CURRENCY

FX drifts from the booked rate

Settlement rate rarely matches the rate at invoice time, and pro-rated ledger entries carry genuine sub-cent residuals.

MERCHANT OF RECORD

Tax was never really yours

The processor remits VAT/sales tax on your behalf — that money must be tracked as a liability, not revenue, before it ever hits cash.

THE GAP BETWEEN TWO WELL-FUNDED CATEGORIES

Close automation exists. This sub-ledger doesn't.

Autonomous-close platforms already handle checklists and flux narratives on top of an ERP. None of them own the messy, underneath layer: turning one processor payout into a proven set of journal entries.

01 Close orchestration exists

Checklist agents, flux-analysis agents, and reconciliation-rule agents are shipping today, on top of an existing ERP.

02 Payout decomposition doesn't

The layer feeding those tools — breaking a merchant-of-record payout into revenue, fees, tax, and disputes — is still done by hand.

03 MoR platforms made it worse

Dodo, Paddle, and similar platforms shifted more of the burden onto the payout itself — net of taxes someone else already remitted.

INTRODUCING

TieOut

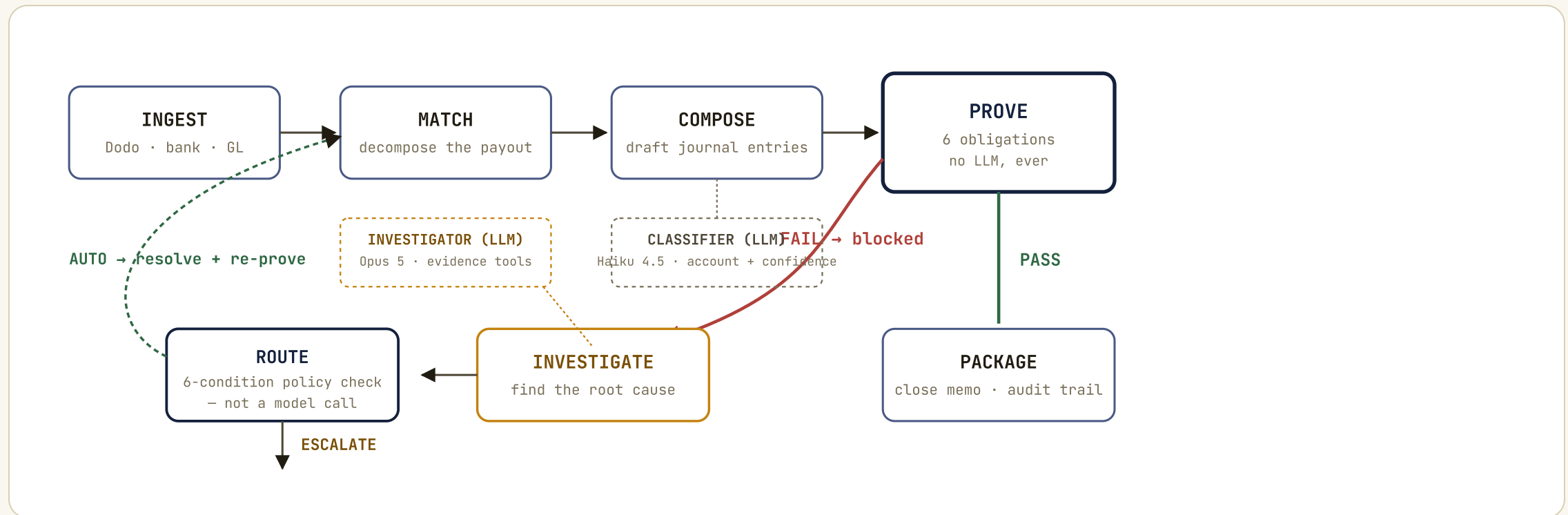
An agent that decomposes settlement payouts, drafts the journal entries, and proves the clearing account is empty before it will let a period close.

PROOF-CARRYING CLOSE

JUDGMENT COMPILER

A STATE MACHINE, NOT A CHATBOT WITH TOOLS

Two narrow places an LLM is ever consulted



Every path back onto the spine passes through **PROVE** again — nothing an LLM touches reaches the ledger unchecked.

TRUST, ENGINEERED — NOT PROMISED

Proof-Carrying Close

A language model never gets to decide the books are correct. Six deterministic proof obligations run on every close — arithmetic checks with no model call involved. If even one fails, the period cannot close. Not a warning. A block.

P3 • CLEARING ACCOUNT ROLLFORWARD — FAILED

```
# the check every autonomous action must pass def
route(exception, remedy, proofs, rules): return "AUTO"
if all([ all(p.passed for p in proofs if p.blocking), #
proofs hold abs(exception.amount) < 250, confidence ≥
0.85, rules.has_match(exception), not
touches_restricted_acct, not rounding_cap_breached, ])
else "ESCALATE"
```

ONE HUMAN DECISION, MADE ONCE

The Judgment Compiler

When a controller resolves an exception, that ruling is compiled into a versioned, reusable rule with a paper trail back to who made the call. The exact same situation clears itself automatically next period.

ESCALATED

Dispute fee + FX never split

Amount: \$4,812.50 — above the \$250 materiality ceiling. Agent proposes a fix but will not apply it alone.

HUMAN RULES ONCE

"Split the fee to 6820, FX to 7410"

One sentence, required — the rationale is the training signal. No blank approvals allowed.

COMPILED

dispute_fee_fx_split v1

Next period, this exact archetype resolves autonomously — no human call needed.

14 TYPED EXCEPTIONS, ONE POLICY

Autonomy is a conjunction, not a vibe

Every exception — duplicate charges, FX variance, unmatched bank lines, low-confidence classifications — runs through the same six-condition check. All six pass, or a human decides. Never the reverse.

- ✓ **Proof still holds after the fix**
The remedy is verified against arithmetic before it's trusted.
- ✓ **Under \$250 materiality**
Size gates autonomy — never confidence alone.
- ✓ **Confidence ≥ 0.85 , archetype known**
Novelty always routes to a human, regardless of dollar size.
- ✓ **No restricted account, cap intact**
The rounding account is hard-capped — an agent can never bury a real discrepancy there.

A NUMBER GOING UP PROVES NOTHING ON ITS OWN

We run a control to isolate the effect of learning

RUN 1 • AUGUST

No rules yet

Baseline automation rate — the system has seen nothing before this run.

RUN 2 • SEPTEMBER

Rules from Run 1 applied

Treatment — the Judgment Compiler's rules are live.

RUN 2 • CONTROL

Same period, rules disabled

The only variable removed — isolates what the learning actually caused.

Ground truth comes from a planted-exception manifest no agent or prompt can read — precision and recall are measured against it, not asserted.

CHOSEN FOR TWO THINGS: NO FLOATS, AND A DEMO THAT SURVIVES A DEAD NETWORK

Tech stack & how we used Agent Orchestrator

BACKEND

Python · FastAPI

MONEY

Decimal / NUMERIC(18,4)

DATABASE

Postgres

AGENTS

Anthropic Messages API

INVESTIGATOR

Opus 5

CLASSIFIER

Haiku 4.5 via TensorMux

FRONTEND

Next.js + shadcn/ui

LIVE FEED

Server-Sent Events

OBSERVABILITY

Neatlogs

DATA SOURCE

Dodo Payments

AO — MANDATORY FROM HOUR 0

Freeze first, fan out second

We froze the database schema and two interfaces in the first two hours. That freeze is what let us run AO sessions in parallel for everything after.

RESULT

~13 concurrent AO sessions

Six sessions built the six proof obligations concurrently; seven more built the exception handlers — each merging only once it passed the shared test suite.

THE QUESTIONS EVERY JUDGE IS ALREADY ASKING

Answered by the product, not the pitch

QUESTION	WHERE THE ANSWER LIVES
Can I trust this with financial operations?	The proof engine blocks the close — an LLM never computes a final dollar amount.
Where is human oversight?	Materiality-gated escalation with a mandatory, non-empty rationale.
What happens after a mistake?	The Judgment Compiler — every ruling becomes a rule with provenance.
Why isn't this just a rules engine?	The decomposition is under-determined; judgment is used to <i>manufacture</i> the deterministic rules.
Could this become a real product?	Runs on top of any processor's payout API — no ERP rip-and-replace required.

Finance teams break one deposit into hundreds of transactions, by hand, every cycle.

*TieOut does it autonomously — but it will not post anything it cannot
prove.*

[Team name] • [contact / repo link] • Track 2 — Autonomous Office of the CFO